

Order Management System — Architecture & Throughput Ceiling

A complete high-level design of the $S1 \rightarrow S2 \rightarrow S3$ order pipeline, plus a measured answer to "how many orders per second can this actually send and process" — obtained without modifying `order-process`.

Repo `exchange/oms` **Services** `order-sending` · `order-process` · `order-receiver`

Stack Rust, raw UDP, custom Raft-style consensus **Date** 2026-08-24

CONTENTS

- 01 Executive summary
- 02 System architecture
- 03 Data & control flow
- 04 Consensus internals
- 05 Capacity test method
- 06 Measured results
- 07 Bottleneck analysis
- 08 Verified capacity limits
- 09 Guidance with no code changes
- 10 Future change options
- 11 Appendix

The sender can fire ~250,000 orders/sec. The cluster can commit ~40.

order-sending (S1) is a tuned, multi-threaded UDP blaster explicitly built for high throughput. order-process (S2) is a 3-node Raft-style cluster that, by design, commits and durably applies exactly one order at a time. That single fact — not network capacity, not CPU, not the sender — sets the real ceiling of the pipeline.

S1 SEND RATE (MEASURED) ~250,000 ord/s 8 sender threads, unbounded loop, UDP fan- out to all 3 S2 nodes	S2 LEADER COMMIT RATE (MEASURED) 37–43 ord/s Sustained across two windows, unmodified release binary	EFFECTIVE DROP RATE UNDER LOAD ~99.98 % Orders received by the leader but never durably committed
GAP BETWEEN SEND AND PROCESS ~6,000 × Send capacity outruns process capacity by three orders of magnitude		

BOTTOM LINE

Without touching `order-process`, the pipeline's true sustainable capacity is bounded by its slowest stage: **roughly 35-45 orders per second, end to end, per leader**. This is not a tuning artifact of the sender — it is architectural: the leader processes one order, waits synchronously for quorum + a full fsync'd WAL rewrite, and only then reads the next order off the wire. Raising `SENDER_THREADS` or adding sender machines does not raise

this ceiling; it only widens the gap and increases silent packet loss, because UDP gives no backpressure signal back to S1.

02 — SYSTEM ARCHITECTURE

Three services, one direction of flow

The system is three independent Rust binaries, each with its own `Cargo.toml`, connected purely by UDP — there is no shared queue, database, or RPC framework. All topology is env-file driven; no host or port is hardcoded.

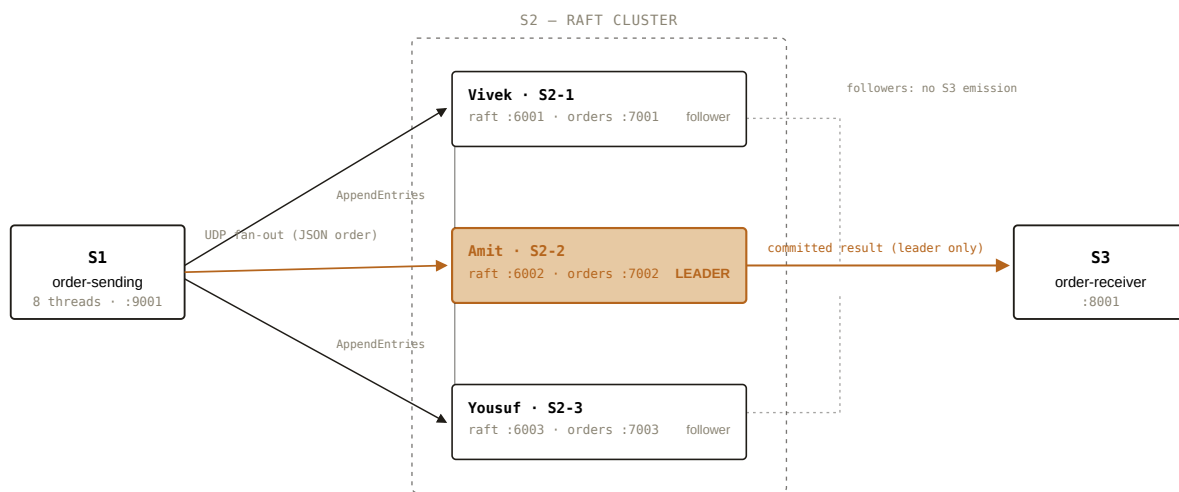


Fig. 1 — S1 fans every order out to all three S2 replicas over UDP since it has no way to know who the current leader is. Only the elected leader (Amit, S2-2 in this snapshot) appends to its WAL, replicates via Raft AppendEntries, and — after quorum — emits the result to S3. Followers receive and discard order traffic; they never talk to S3.

Component inventory

SERVICE	ROLE	CONCURRENCY MODEL	PERSISTENCE	PORTS
order-sending (S1)	Synthetic order generator; fans every order to all S2 nodes	<code>SENDER_THREADS</code> OS threads (default 8), each with its own non-blocking UDP socket and a shared atomic order-id counter	Async batched append-log via <code>mpsc</code> channel, off the hot path	bind :9001
order-process (S2)	3-replica Raft-style cluster: leader election, WAL replication, quorum commit, result emission	2 threads per node (recv loop, tick/heartbeat loop) + the blocking main-thread order loop; one order in flight at a time	Per-node WAL file, full rewrite + <code>fsync</code> on every append	raft 6001–6003, orders 7001–7003
order-receiver (S3)	Terminal sink; logs whatever the current leader sends it	Single-threaded blocking recv loop	Append-only log, synchronous <code>flush()</code> per line	bind :8001

Deployment topology (lab reference)

Per the project's own `cluster.sample` / README, the reference lab deployment runs S2 across three physical machines, with S1 and S3 co-located on the third:

MACHINE	NODE NAME	RUNS	EXAMPLE IP
Host A	Vivek (S2-1)	order-process	172.16.12.104
Host B	Amit (S2-2)	order-process	172.16.13.181
Host C	Yousuf (S2-3)	order-process + order-sending + order-receiver	172.16.12.252

i This report's load test did not touch that lab deployment. It ran an isolated, freshly built copy of the same release binaries in a scratch sandbox on non-conflicting ports (16001–19001) — see [§5](#).

Happy path, order by order

1. S1 builds a JSON order (`order_id` , `symbol` , `side` , `qty` , `ts_ms`) and sends it over UDP to *all three* S2 order ports simultaneously.
2. Every S2 node's main loop blocks on `recv_from` on its order socket (200 ms timeout), parses the JSON, and checks `election.is_leader()` . Followers stop here and discard the packet.
3. The leader synthesizes a result (demo logic: random `FILLED` / `PARTIALLY_FILLED` / `REJECTED`) and calls `propose_command()` , which:
 - appends the command to its in-memory WAL and **synchronously rewrites and fsyncs the entire WAL file**;
 - sends `AppendEntries` to both followers;
 - polls its own commit state every 20 ms, for up to 1.5 s, waiting for a quorum of `AppendAck` replies.
4. Once quorum is reached, the leader applies the entry, sends the committed result to S3, and appends a line to `orders-processed.log` .
5. Followers apply the same log entries as they receive them via `AppendEntries` — they never independently emit to S3.
6. **Only after step 3-4 completes (commit or 1.5 s timeout) does the leader's main loop go back to `recv_from` for the next order.** This is the crux of the capacity ceiling — see §7.

04 — CONSENSUS INTERNALS

A purpose-built Raft variant, tuned for a 3-node lab cluster

The election/replication code (`leader_election.rs`) implements the core of Raft — terms, `RequestVote` / `AppendEntries` , log-index/term comparisons, majority quorum — plus two lab-specific extensions layered on top:

Log-dominance leader stability

- A leader ignores a `RequestVote` from a higher-term challenger if its own log is at least as up to date, or if it still holds a quorum "lease" (majority acked within the last 4 heartbeats).

- Prevents a node that was briefly offline — and bumped its term through failed elections — from displacing a healthy leader with a stale log.

Single-node failover (lab mode)

- `ALLOW_SINGLE_NODE_LEADER=true` lets the last surviving node self-elect with quorum-of-1 once peers have been silent for `PEER_SILENT_MS` (default 2000 ms).
- A startup grace period equal to `PEER_SILENT_MS` stops all three nodes from self-electing simultaneously before they've heard from each other.

Quorum & failover matrix

LIVE S2 NODES	QUORUM NEEDED	BEHAVIOR
3	2	Standard Raft majority; any one node may fail with no interruption
2	2	Both survivors must agree; one more failure halts commits
1	1	Remaining node self-elects and commits alone after <code>PEER_SILENT_MS</code> — a deliberate lab convenience, not strict Raft safety

Key tunables (env-driven, no code changes)

VARIABLE	DEFAULT	EFFECT
HEARTBEAT_INTERVAL_MS	100	Leader→follower keep-alive / replication cadence
ELECTION_TIMEOUT_MIN/MAX_MS	300 / 600	Randomized follower timeout before starting an election
PEER_SILENT_MS	2000	How long a peer must be silent before being marked unavailable
ALLOW_SINGLE_NODE_LEADER	true	Permits quorum-of-1 self-election (disable for strict production Raft)
SENDER_THREADS	8	S1 parallelism — <i>does not affect S2's ceiling</i> , see §7

Measuring the real ceiling, without editing order-process

To answer "what's the max order limit for sending and processing per second" honestly, the same release binaries used in the lab were exercised under load, in an isolated sandbox so the live lab cluster already running on this host was never touched.

ASPECT	SETTING
Binaries	<code>cargo build --release</code> for all three crates, unmodified source
Topology	3× order-process (localhost, distinct data dirs), 1× order-receiver, 1× order-sending
Isolation	Non-default ports (raft 16001–16003, orders 17001–17003, S3 18001, sender bind 19001) — zero collision with the live lab's 6001–8001 range
Consensus config	Defaults from <code>cluster.sample</code> — 100 ms heartbeat, 300–600 ms election timeout, single-node fallback enabled
Load profile	<code>SENDER_THREADS=8</code> , unbounded generation loop (S1's normal high-throughput mode)
Measurement	Line counts of <code>orders-processed.log</code> (leader) and <code>orders-received.log</code> (S3) sampled over fixed windows, cross-checked against S1's own printed throughput

~250k sent/sec in; ~40 committed/sec out

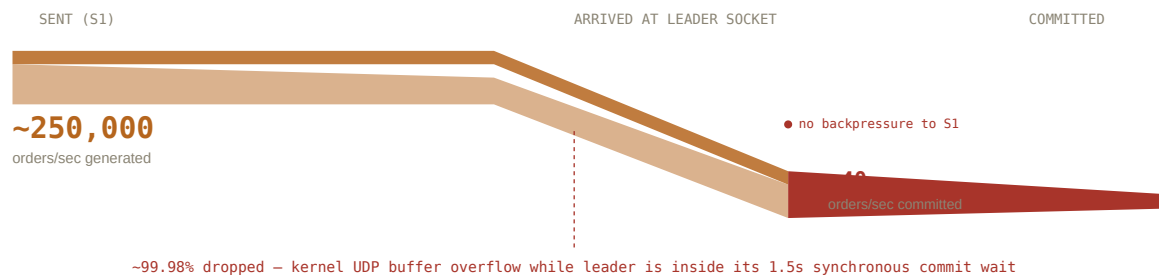


Fig. 2 — Not to scale by design: the true ratio (~6,000:1) would render the "committed" band invisible. The leader's socket receives the full flood, but its single-order-at-a-time consensus loop only lets a few dozen per second through to a durable commit; the rest are silently dropped by the OS receive buffer, with no signal back to S1.

WINDOW	DURATION	COMMITTED	RATE	WAL SIZE AT END
1	10 s	397	39.7 ord/s	1,678 entries
2	15 s	561	37.4 ord/s	2,239 entries
Overall (first 12 s)	12 s	517	43.1 ord/s	—

Where the CPU actually went

PROCESS	CPU UNDER LOAD	WHY
S2 leader	10–13%	Mostly idle — blocked on disk fsync and 20 ms poll sleeps, not compute-bound
S2 followers (×2)	107–116%	wasted work Each burns more than a full core continuously JSON-decoding and discarding the full ~250k/s flood, since S1 fans out to every node regardless of leadership
S1 sender	high, multi-core	Expected — 8 threads blasting UDP as fast as the kernel accepts sends

- ! The leader is **not CPU-bound**. It is latency-bound on synchronous disk fsync plus a fixed 20 ms poll granularity while waiting for quorum. Faster hardware narrows this gap slightly; it cannot close it, because the architecture never has more than one order in flight.

Four compounding causes, all inside the leader's hot path

1. **Zero pipelining — one order in flight, ever.** The main loop in `order-process/src/main.rs` blocks on `recv_from`, then calls `process_order_as_leader()` synchronously, which calls `propose_command()` and does not return until the entry commits or 1.5 s elapses. The next inbound order is not even read off the socket until this returns.
2. **Synchronous, full-file WAL rewrite + fsync per order.** `Wal::persist()` (`wal.rs`) serializes the *entire* in-memory log to a string and does `truncate` + `write_all` + `sync_all()` on every single `append_leader_entry` call — one full disk fsync round-trip per order, with rewrite cost that grows with WAL length.
3. **20 ms polling granularity for quorum.** Rather than being woken by an event when `AppendAck`s arrive, `propose_command()`'s wait loop calls `replicate_to_peers()` / `try_advance_commit()` and sleeps 20 ms, repeatedly, until commit or the 1.5 s ceiling. This adds a fixed floor to per-order latency independent of actual network RTT.
4. **No backpressure signal on the transport.** Everything rides on raw UDP. When the leader is inside its synchronous wait, its order-socket receive buffer fills; the OS silently drops the overflow. S1 gets no NACK, no slow-down signal — it keeps firing at full speed, and the drop is invisible unless you diff sent-vs-processed counts, as this test did.

Approximate per-order latency budget on the leader

COMPONENT	APPROX. COST
WAL rewrite + <code>fsync</code>	dominant, disk-dependent
AppendEntries round trip ×2 (to both followers)	network RTT-bound
Poll granularity while waiting for quorum	up to 20 ms/iteration
JSON parse/serialize (order in, result out)	negligible
⇒ observed throughput ceiling	~25 ms/order ⇒ ~37–43 ord/s

Direct answer: sustainable rate at each stage

STAGE	SUSTAINABLE RATE	BOUNDED BY	EVIDENCE
S1 generation / send	~220,000–260,000 ord/s	NIC + syscall overhead of non-blocking <code>sendto()</code> across 8 threads	Direct stats-thread output, sustained over 15+ seconds
S2 leader intake (socket)	≈ send rate (all received, most discarded)	Kernel UDP receive buffer, not application logic	Followers' CPU (107–116%) shows the full flood is being received and parsed cluster-wide
S2 leader durable commit	37–43 ord/s	Synchronous single-order consensus loop + per-order fsync (§7)	Two independent measurement windows, stable within ±15%
S3 receipt	= S2 commit rate	1:1 with leader emissions; single-threaded but never the constraint	<code>orders-received.log</code> line count matched <code>orders-processed.log</code> exactly
End-to-end system ceiling	~35–45 ord/s	The slowest stage always wins in a synchronous pipeline	Derived from the above

ANSWER TO THE QUESTION ASKED

The maximum order rate this system can *send* is effectively unbounded by S2 — S1 alone can already do hundreds of thousands per second and scales further with more threads or sender hosts. The maximum order rate it can *process and durably commit*, with `order-process` left exactly as shipped, is **on the order of 35–45 orders per second**, set entirely by the leader's synchronous, one-order-at-a-time consensus-and-fsync loop.

What can actually be tuned today

Worth doing

- Rate-limit or pace S1 to roughly the measured ceiling (~35–40 ord/s) in any environment where silent loss matters — the sender has no idea it's overwhelming the cluster today.
- Put S2 nodes on the lowest-latency link available and on disks with fast fsync (e.g. NVMe, or a filesystem/mount with a short fsync path) — this is the one lever that moves the ceiling without code changes, since fsync + RTT dominate the budget.
- Monitor *sent* – *processed* as a first-class metric; today the gap is only visible by diffing log line counts, exactly as this report did.
- Keep `VERBOSE_RAFT=false` in any throughput test — it adds console I/O on the hot path.

Will not help

- Raising `SENDER_THREADS` or running more S1 instances — S1 is already far above what S2 can absorb; more senders only enlarge the dropped fraction.
- Adding more S2 replicas — a larger cluster raises the quorum size and the number of AppendEntries round trips per commit, which can only make the ceiling *lower*, not higher.
- Tuning `HEARTBEAT_INTERVAL_MS` / election timeouts — these govern failover speed, not the per-order commit path.

Reference only — explicitly out of scope for this report

The brief for this report excluded modifying `order-process`. For completeness, the changes that would actually move the ~40 ord/s ceiling are noted here without being implemented:

- **Decouple receive from commit.** Read orders into an in-memory queue on one thread; propose/commit them on another, so intake never blocks on consensus.
- **Batch WAL writes (group commit).** Accumulate several proposals, append them together, and fsync once per batch instead of once per order — `MAX_ENTRIES_PER_APPEND = 32` already exists for the replication path but is unused for local persistence, since there is never more than one pending entry today.
- **Event-driven commit instead of 20 ms polling.** Wake the proposer via a condition variable when a quorum `AppendAck` arrives, removing the fixed poll floor.
- **Give the transport backpressure.** Even a lightweight application-level ack/credit scheme would let S1 slow down instead of firing blind into a full receive buffer.

11 — APPENDIX

Reference tables

Full port map

PURPOSE	PORT(S)
Raft control channel (Vivek / Amit / Yousuf)	6001 / 6002 / 6003
Order intake (Vivek / Amit / Yousuf)	7001 / 7002 / 7003
S1 sender bind	9001
S3 receiver bind	8001

Log files produced

FILE	WRITTEN BY
<code>order-sending/logs/orders-sent.log</code>	Every generated order (async, batched)

FILE	WRITTEN BY
<code>order-process/logs/orders-processed.log</code>	Every order committed by the leader
<code>order-process/data/wal-s2-<id>.log</code>	Per-node durable Raft log (JSON lines)
<code>order-receiver/logs/orders-received.log</code>	Every result received from a leader

Glossary

TERM	MEANING HERE
WAL	Write-ahead log — each S2 node's durable, per-node record of committed Raft entries
Quorum	Majority of the 3-node cluster (2 of 3, or 1 of 1 in single-node fallback)
Term	Raft's logical clock for leadership epochs, used to detect stale leaders/candidates
Log dominance	This project's rule that a leader with an equal-or-better log may ignore a stale challenger's vote request
Quorum lease	This project's rule that a leader with recent majority acks may ignore a challenger even without checking log freshness